Part 3: Critical Reflection Report

Individual Contribution

Eryk – I have contributed to 40% of the project, I made the backend API in Spring Boot for Part 1 and answered Q3. Security Standards in the Critical Reflection Report.

Haries Sathyamurthy: I contributed 30% to the project. I contributed by fixing the defect issue, I also implemented the viewModel and Rotation Handling. I also worked on debugging errors found in the code as well as working on user interface such as making the report button send the user directly to the mail app. I also answered Q1 . Architecture in the Critical Reflection Report.

Isher: I contributed 30% to the project, by implementing add check in Part 2, the main layout of the app and the local database, as well as answering Q2. Data Privacy in the Critical Reflection Report.

Architecture

We used a repository for android, so it handles all the data and to manage organisation in the app. So, the repository is what works between the viewModel and the data sources, such as the local database. In the project, the viewModel does not directly access the database, but instead calls the repository which then performs the data operations for it. This is useful because it separates the user interface from the data logic which makes the app easier to manage and update, this is also referred to as a Single Source of Truth (SSOT), with the repository being the one place the rest of the app gets data from. The same idea appeared in Lab 6, where our Spring Boot REST API used a JpaRepository so that the controller never had to deal directly with SQL or the database, it just called methods on the repository.

If the app was connected to an API later, the repository would manage both the local database and the server. It would decide when to get data from the internet and when to use the data that is saved on the device. This makes sure the app works even with no internet connection. Usually, data is loaded from the local database, simultaneously the repository updates it in the background when new data is fetched from the server.

When handling offline sync conflicts, the repository would require simple rules. One way would be to use timestamps, so each item would store when it was last updated. When syncing, the app compares the server version with the local version. If the local version is newer, it gets sent to the server, however, if the server version is newer, it replaces the local one. This approach is called Last-Write-Wins (LWW) where whichever version was edited most recently is kept. It's a simple rule but it has weaknesses, for example if the device's clock is wrong the timestamps can't be trusted, or if both the server and the local copy were edited since the last sync, neither change should really be discarded. The app is also able to mark changes that haven't been synced, so they can be uploaded later. This helps keep the data the same on both the device and the server.

Data Privacy

All data stored in an unencrypted SQLite database is written on the device's file system in plain text. This includes sensitive information such as vehicle identification numbers (VINs), owner details, location history or maintenance records. If an attacker gains access to the file, they can read and extract all this information without having to bypass any additional security measures.

On a standard (non-rooted) Android device, this risk is somewhat mitigated by Android's App Sandbox, a feature which isolates each application from others. Every app has its own unique Linux user ID (UID), and its data is stored in a private directory. Normally, other apps cannot access this directory due to strict file permissions enforced by the operating system. This means that even if a malicious app is installed, it shouldn't read another app's SQLite database. We saw a small piece of this isolation in Lab 11, where the SecondActivity in the AndroidManifest was declared android:exported="false" so that no other app could launch it.

However, this protection relies heavily on the integrity of the operating system. When a device is rooted, these protections break down. Rooting grants 'superuser access' to the device, bypassing the restrictions created by the App Sandbox. With root privileges, an attacker, or a malicious app granted root access, can access any part of the file system, including the private directories of other apps. They can navigate to the app's data folder and copy the SQLite file directly, then open it with built-in tools and see all the data immediately. Rooted devices also frequently lose manufacturer security updates, leaving known vulnerabilities unpatched and increasing the likelihood of attacks.

Even on a non-rooted device, an attacker with physical access can attempt extraction by booting into a custom recovery or using debugging channels such as ADB (Android Debug Bridge). Modern Android devices have encryption and secure boot mechanisms to ensure confidentiality but on a rooted device these protections can be disabled, leaving the file system exposed.

To mitigate these risks, developers should never rely on the App Sandbox alone to protect sensitive data. Instead, they should implement additional safeguards, such as encrypting the database with SQLCipher, storing keys in the Android Keystore and limiting the amount of sensitive data stored on the device.

Security Standards

Modern browsers show a 'Not Secure' warning or a struck-through padlock for self-signed certificates such as in part 1, as the browsers cannot verify the authenticity and identity of the owner who has issued and signed it. Nowadays, certificates need to be issued by a trusted Certificate Authority (CA), in the form of TLS certificates. These ensure that communication between the client and server is protected as well as identifying who the traffic from the user is being sent to. Without CA verification, an attacker could generate their own self-signed certificate claiming to be legitimate and intercept traffic, i.e., a Man-in-the-Middle (MITM) attack and the user would have no way to detect the impersonation.

When you visit a website, the site will send a digital certificate to your browser, the browser will then compare the issuer with a list of trusted Certificate Authorities (Root CA). If a match can't be found, the client browser checks to see whether a trusted Root CA signs the issuing CA certificate. 'The browser's chaining engine continues verifying the issuer of each certificate until it finds a trusted root or upon reaching the end of the trust chain' (Gaff, 2020). If it reaches the end without being able to verify the issuer, the browser will warn the end user of a security risk upon entering the website. Trust within the chain is enforced cryptographically, this means each CA signs the certificate below it using its private key, and the browser verifies that signature using the CA's public key, which is pre-installed in the operating system's truststore. Tampering with any certificate in the chain invalidates the signature.

This difference between certificates becomes clear when comparing our Part 1 backend with the Dog CEO API consumed in Lab 5, https://dog.ceo/api/breeds/image/random, where the Dog CEO API uses a certificate signed by a publicly trusted CA whose root certificate is already in Java's default truststore. Whereas the backend in Part 1 uses a self-signed certificate, so a client like RestTemplate would refuse to connect with an SSL handshake error unless the certificate were manually added to the truststore.

References

- *NEWSWIRE (2025). Why You Should Use the Repository Pattern in Android #Kotlin #Android. [online] Medium. Available at: https://medium.com/@madhav2002/why-you-should-use-the-repository-pattern-in-android-232c73c963ef*
- *Gaff, T. (2020). What is the Certificate Chain of Trust? [online] Keyfactor. Available at: https://www.keyfactor.com/blog/certificate-chain-of-trust/.*